TCC70xx MCU BSP

API Specification for Sound Processing Unit

Rev. 0.10 [A]

2021-09-21

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips, Inc.

Kindly visit www.telechips.com for more information.

TABLE OF CONTENTS

Contents

Figures

Tables

1 INTRODUCTION

This document describes the sound processing unit device driver (hereinafter referred to as SPU driver) of TCC70xx MCU BSP. For more detailed information on Sound Processing Unit (SPU) in the TCC70xx MCU Sub-system, refer to Chapter 16. Audio Interface Controller & Sound Processing Unit in *"TCC70xx Full Specification"*[1].

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

SPU	Sound Processing Unit
WG	Waveforms Generator
SSL	Sound Source Loader
AVC	Audio Volume Controller
SRC	Sample Rate Converter
TAS	Telechips Audio Stream

3 SOURCE FILES

3.1 Location of Source Files

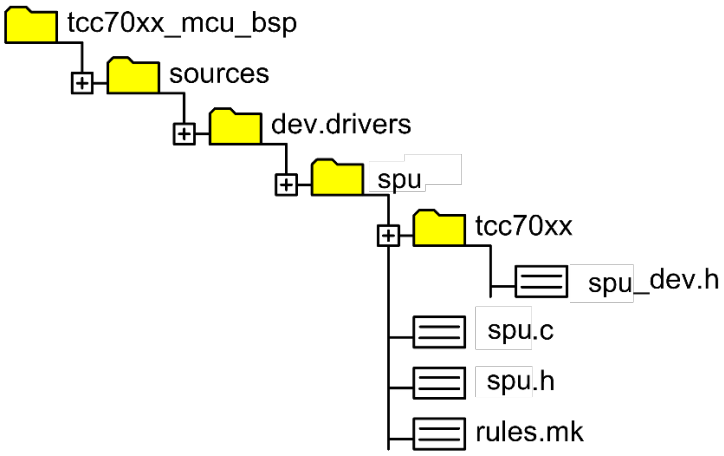

Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- spu_dev.h
  - Contains SPU register information
- spu.c
  - SPU device driver source code
- spu.h
  - SPU common header including API & Defines
- rules.mk
  - Rules used for build

4 CONSTANTS

4.1 Driver Operation Values

Declaration

```
#define SPU_OK                          0
#define SPU_INVALID_ERR                 -1

#define SPU_DISABLE                     0UL
#define SPU_ENABLE                      1UL
```

Description

Value	*Description*
SPU_OK	*Success*
SPU_INVALID_ERR	*Fail*
SPU_DISABLE	*Disable SPU device*
SPU_ENABLE	*Enable SPU device*

Remark

None

4.2 Number of SPU Channel

Constant values for the number of SPU channels

Declaration

```
#define MAX_NUM_OF_SSL          5UL
#define MAX_NUM_OF_WG           5UL
#define MAX_NUM_OF_SRC          6UL
#define MAX_NUM_OF_AVC          16UL
```

Description

Value	*Description*
MAX_NUM_OF_SSL	*The number of total SSL, SSL0 to SSL4*
MAX_NUM_OF_WG	*The number of total WG, WG0 to WG4*
MAX_NUM_OF_SRC	*The number of total SRC, SRC0 to SRC5*
MAX_NUM_OF_AVC	*The number of total AVC, AVC0 to AVC15, AVC15 (Zero Output)*

Remark

The output of avc_ch15 is zero data.

4.3 AVC Control Value

Constant values for the AVC control

Declaration

```
#define AVC_BYPASS                  0UL
#define AVC_VOL_CTL                 1UL

#define AVC_UNMUTE                  0UL
#define AVC_MUTE                    1UL
```

Description

Value	Description
AVC_BYPASS	Set the AVC channel to bypass mode
AVC_VOL_CTL	Set the AVC channel to volume control mode
AVC_UNMUTE	Set the AVC channel to unmute mode
AVC_MUTE	Set the AVC channel to mute mode

Remark

None

4.4 SRC Control Value

Constant values for the SRC control

Declaration

```
#define SRC_SRC_MODE                0UL
#define SRC_BYPASS_MODE             1UL
```

Description

Value	Description
SRC_SRC_MODE	Set the SRC channel to sample-rate convert mode
SRC_BYPASS_MODE	Set the SRC channel to bypass mode

Remark

None

4.5 SSL Control Value

Constant values for the SSL control

Declaration

```
#define SSL_MONO                    0UL
#define SSL_STEREO                  1UL

#define SSL_FORCE_STOP              1UL
#define SSL_STABLE_STOP             0UL

#define SSL_IRQ_MASKED              0UL
#define SSL_IRQ_PASSED              1UL

#define SSL_IRQ_STATUS_CLEAR        1UL
#define SSL_IRQ_NO_ACTION           0UL
```

Description

Value	Description
SSL_MONO	Sound source is mono.
SSL_STEREO	Sound source is stereo.
SSL_FORCE_STOP	Stop immediately
SSL_STABLE_STOP	Stop when the current sound loading is finished.
SSL_IRQ_MASKED	Disable SSL interrupt
SSL_IRQ_PASSED	Enable SSL interrupt
SSL_IRQ_STATUS_CLEAR	Clear IRQ status
SSL_IRQ_NO_ACTION	No Action on the IRQ control register.

Remark

None

5 STRUCTURES

5.1 WGConfig_t

Structure to store the configuration values of WG channel

```
typedef struct WGConfig
{
    double          dSampleOverfreq;
    uint32          uiWaveType;
    uint32          uiIdleTimeMS;
    uint32          uiAttackTimeMS;
    uint32          uiStableTimeMS;
    uint32          uiReleaseTimeMS;
    uint32          uiFilterType;
    uint32          uiTonegenRepeat;
} WGConfig_t;
```

Description

Value	Description
dSampleOverfreq	The target frequency of waveform. When the target frequency is determined, write the value of 48000 / target frequency.
uiWaveType	Waveform type. Use enumeration variable, WG_WAVE_TYPE_t.
uiIdleTimeMS	Idle time(msec) value. The amount of time to wait before generating a wave.
uiAttackTimeMS	Attack time(msec) value. Fade-in time that starts immediately after idle time.
uiStableTimeMS	Stable time(msec) value. Constant DB is used while stable time is running.
uiReleaseTimeMS	Release time(msec) value. Fade-out time that starts immediately after stable time.
uiFilterType	Filter Type. 0x0: Bypass, 0x1: 8 kHz, 0x2: 5 kHz, 0x3: 2 kHz cutoff lowpass filter.
uiTonegenRepeat	Repeat wave generation 0x0: Infinite mode, other values: the number of iterations.

Remark

Resolution of the dSampleOverfreq is 0.125.

5.2 SSLConfig_t

Structure to store the configuration values of SSL channel

Declaration

```
typedef struct SSLConfig
{
    uint32          uiMode;
    uint32          uiRepeat;
    uint32          uiFifoAddr;
    uint32          uiFifoSize;
    uint32          uiDummySize;
} SSLConfig_t;
```

Description

Value	Description
uiMode	If sound source is mono, set uiMode to SSL_MONO. If sound source is stereo, set uiMode to SSL STEREO.
uiRepeat	Number of repetitions. 0: Infinite mode, others: the number of iterations.
uiFifoAddr	The address of sound source. It must be a multiple of 16.
uiFifoSize	The size of sound source. It must be a multiple of 16.
uiDummySize	The size of dummy data. It must be a multiple of 4.

Remark

None

5.3 SRCConfig_t

Structure to store the configuration values of SRC channel

Declaration

```
typedef struct SRCConfig
{
    uint32          uiByPass;
    uint32          uiFifoSetPoint;
    uint32          uiInitZeroSize;
    double          dRatio;
} SRCConfig_t;
```

Description

Value	Description
uiByPass	Select sample-rate converting or bypass
uiFifoSetPoint	Optimum buffer level for SRC core
uiInitZeroSize	Set zero padding size for FIR SRC core
dRatio	The value for sample-rate converting ratio. If dRatio is (6.0 / 1.0), it means x6 up-sampling. If dRatio is (1.0 / 2.0), it means 1/2 down-sampling.

Remark

None

5.4 AVCConfig_t

Structure to store the configuration values of AVC channel.

Declaration

```
typedef struct SSLConfig
{
    uint32          uiMode;
    uint32          uiPeriod;
    uint32          uiInterval;
    uint32          uiWait;
    uint32          uiGain;
} SRCConfig_t;
```

Description

Value	Description
uiMode	Select bypass mode or volume control mode
uiPeriod	The number of volume changes.
uiInterval	The number of frames applied to the volume change
uiWait	The number of frames to wait before volume change
uiGain	The volume gain of channel

Remark

- Channel gain value is signed data and 2's compliment is used for express negative value.
- Resolution of channel gain value is 0.0625 and the expression boundary is -64[dB] to 63.9375[dB].

6 ENUMERATIONS

6.1 SPU_SSL_CH_t

Enumerate variables of SSL channel index. The SSL channel is selected by this enumeration variables.

Declaration

```
typedef enum SPU_SSL_CH {
    SPU_SSL_0               = 0UL,
    SPU_SSL_1               = 1UL,
    SPU_SSL_2               = 2UL,
    SPU_SSL_3               = 3UL,
    SPU_SSL_4               = 4UL
} SPU_SSL_CH_t;
```

Description

Value	Description
SPU_SSL_0	Sound Source Loader channel index 0
SPU_SSL_1	Sound Source Loader channel index 1
SPU_SSL_2	Sound Source Loader channel index 2
SPU_SSL_3	Sound Source Loader channel index 3
SPU_SSL_4	Sound Source Loader channel index 4

Remark

None

6.2 SPU_WG_CH_t

Enumerate variables of Wave Generator (WG) channel index. The WG channel is selected by this enumeration variables.

Declaration

```
typedef enum SPU_WG_CH {
    SPU_WG_0                = 0UL,
    SPU_WG_1                = 1UL,
    SPU_WG_2                = 2UL,
    SPU_WG_3                = 3UL,
    SPU_WG_4                = 4UL
} SPU_WG_CH_t;
```

Description

Value	Description
SPU_WG_0	Wave Generator 0
SPU_WG_1	Wave Generator 1
SPU_WG_2	Wave Generator 2
SPU_WG_3	Wave Generator 3
SPU_WG_4	Wave Generator 4

Remark

None

6.3 SPU_AVC_CH_t

Enumerate variables of AVC channel index. The AVC channel is selected by this enumeration variables.

Declaration

```
typedef enum SPU_AVC_CH {
    SPU_AVC_WG0             = 0UL,
    SPU_AVC_WG1             = 1UL,
    SPU_AVC_WG2             = 2UL,
    SPU_AVC_WG3             = 3UL,
    SPU_AVC_WG4             = 4UL,
    SPU_AVC_SSL0            = 5UL,
    SPU_AVC_SSL1            = 7UL,
    SPU_AVC_SSL2            = 9UL,
    SPU_AVC_SSL3            = 11UL,
    SPU_AVC_SSL4            = 13UL,
    SPU_AVC_MAX             = 15UL
} SPU_AVC_CH_t;
```

Description

Value	Description
SPU_AVC_WG0	AVC channel index 0, WG0 generates mono TAS.
SPU_AVC_WG1	AVC channel index 1, WG1 generates mono TAS.
SPU_AVC_WG2	AVC channel index 2, WG2 generates mono TAS.
SPU_AVC_WG3	AVC channel index 3, WG3 generates mono TAS.
SPU_AVC_WG4	AVC channel index 4, WG4 generates mono TAS.
SPU_AVC_SSL0	AVC channel index 5 and 6, SSL0 generates stereo TAS.
SPU_AVC_SSL1	AVC channel index 7 and 8, SSL1 generates stereo TAS.
SPU_AVC_SSL2	AVC channel index 9 and 10, SSL2 generates stereo TAS.
SPU_AVC_SSL3	AVC channel index 11 and 12, SSL3 generates stereo TAS.
SPU_AVC_SSL4	AVC channel index 13 and 14, SSL4 generates stereo TAS.
SPU_AVC_MAX	Not available

Remark

None

6.4 SSL_STS_t

Enumerate variables of SSL status. This enumeration variables indicate the status of SSL.

Declaration

```
typedef enum SSL_STS {
    SSL_STS_IDLE          = 0UL,
    SSL_STS_DATA          = 1UL,
    SSL_STS_DUMMY         = 2UL,
    SSL_STS_FINAL         = 3UL
} SSL_STS_t;
```

Description

Value	Description
SSL_STS_IDLE	SSL IDLE state, SSL do not anything.
SSL_STS_DATA	SSL Data state, SSL is loading a sound source.
SSL_STS_DUMMY	SSL Dummy state, SSL is loading dummy data.
SSL_STS_FINAL	SSL Final state, SSL waits for stable stop. When the sound source is completely loaded, SSL goes into IDLE state.

Remark

None

6.5 WG_WAVE_TYPE_t

Enumerate variables of the type of waveforms.

Declaration

```
typedef enum WG_WAVE_TYPE {
    WG_SINE_WAVE          = 0UL,
    WG_SAWTOOTH_WAVE      = 1UL,
    WG_SQUARE_WAVE        = 2UL
} WG_WAVE_TYPE_t;
```

Description

Value	Description
WG_SINE_WAVE	WG generates a sine waveform.
WG_SAWTOOTH_WAVE	WG generates a saw waveform.
WG_SQUARE_WAVE	WG generates a square waveform.

Remark

None

7 APIs

7.1 SPU_Init

Initializes SPU. The following functions are executed.

- Clear mixer control register.
- Enable all channel of WG.
- Set global SSL to enable.
- Initialize SRC and AVC.
- Configure SRC ch5 for final TAS.

Declaration

```
void SPU_Init
(
    SRCConfig_t*                    psSRCConfig
);
```

Parameters

Value	Description
psSRCConfig	Pointer value of SRC configuration structure

Return

None

Remark

None

7.2 SPU_DeInit

Disables SPU. The following functions are executed.

- Disable all WG.
- Set global SSL to enable.
- Disable all channel of the SRC.

Declaration

```
void SPU_DeInit
(
    void
);
```

Parameters

None

Return

None

Remark

None

7.3 SPU_Start_WG_Chain

Starts WG Chain. The following functions are executed.

- Select WG channel
- Configure WG channel and start to generate waveforms.
- If AVC is not set to bypass mode for selected channel, configure AVC and start to control channel volume.
- Add the selected WG channel to TAS.

Declaration

```
void SPU_Start_WG_Chain
(
    SPU_WG_CH_t                 uiNth,
    WGConfig_t*                 psWgConfig,
    AVCConfig_t*                psAVCConfig
);
```

Parameters

Value	Description
uiNth	The number of WG channels to stop.
psWgConfig	Ppointer value of WG configuration structure.
psAVCConfig	Pointer value of AVC configuration structure.

Return

None

Remark

None

7.4 SPU_Stop_WG_Chain

Stops WG Chain. The following functions are executed.

- Stop to generate waveforms.
- Clear parameter and dB of AVC channel.
- Set AVC to bypass mode.
- Exclude WG channel from TAS.

Declaration

```
void SPU_Stop_WG_Chain
(
    SPU_WG_CH_t                     uiNth
);
```

Parameters

Value	Description
uiNth	The number of WG channel to stop.

Return

None

Remark

None

7.5 SPU_Start_SSL_Chain

Starts SSL Chain. The following functions are executed.

- Configure SSL and Start SSL
- Configure SRC and Enable SRC
- If AVC is set to volume control mode for selected channel, configure AVC and start to control channel volume.
- Add the selected SSL channel to TAS.

Declaration

```
void SPU_Start_SSL_Chain
(
    SPU_SSL_CH_t                    uiNth,
    SSLConfig_t*                    psSSLConfig,
    SRCConfig_t*                    psSRCConfig,
    AVCConfig_t*                    psAVCConfig
);
```

Parameters

Value	Description
uiNth	The number of SSL channel to start.
psSSLConfig	The pointer value of SSL configuration structure.
psSRCConfig	The pointer value of SRC configuration structure.
psAVCConfig	The pointer value of AVC configuration structure.

Return

None

Remark

None

7.6 SPU_Stop_SSL_Chain

Stops SSL Chain. The following functions are executed.

- If uiForce is 1, stop to load the sound source immediately. This is called a forced stop.
- If uiForce is 0, stop to load the sound source at the end of data. This is called a stable stop.
- Clear parameter and dB of AVC channel.
- Set AVC to bypass mode.
- Exclude WG channel from TAS.

Declaration

```
void SPU_Stop_SSL_Chain
(
    SPU_SSL_CH_t                    uiNth,
    uint32                          uiForced
);
```

Parameters

Value	Description
uiNth	The number of SSL channel to stop.
uiForced	If uiForced is 1, the SSL channel is stopped immediately.

Return

None

Remark

None

7.7 SPU_Set_Volume_Controller

Controls volume. The following functions are executed.

- If AVC is set to volume control mode, the selected channel, configures AVC and starts to control channel volume.
- If AVC is set to bypass mode, the selected channel is into bypass mode.

Declaration

```
void SPU_Set_Volume_Controller
(
    SPU_AVC_CH_t                    uiAvcCH,
    AVCConfig_t*                    psAVCConfig
);
```

Parameters

Value	Description
uiAvcCH	The number of AVC channel to control volume.
psAVCConfig	Pointer value of AVC configuration structure.

Return

None

Remark

None

7.8 SPU_Set_Channel_Mute

Mutes Channel. he following function is executed.

- Set the selected channel to mute or unmute.

Declaration

```
void SPU_Set_Channel_Mute
(
    SPU_AVC_CH_t                    uiAvcCH,
    unsigned int                    uiMute
);
```

Parameters

Value	Description
uiAvcCH	The number of AVC channel to control channel mute.
uiMute	If uiMite is set to AVC_MUTE, the channel goes into mute mode.

Return

None

Remark

None

8 How to use SPU Driver

This device driver can control SPU device to generate a new sound by mixing the waveforms generated by internal WG and the sound source read from the external memory.

For more information, refer to the description of SPU in "*TCC70xx Full Specification*". [1]

8.1 Operation of SPU

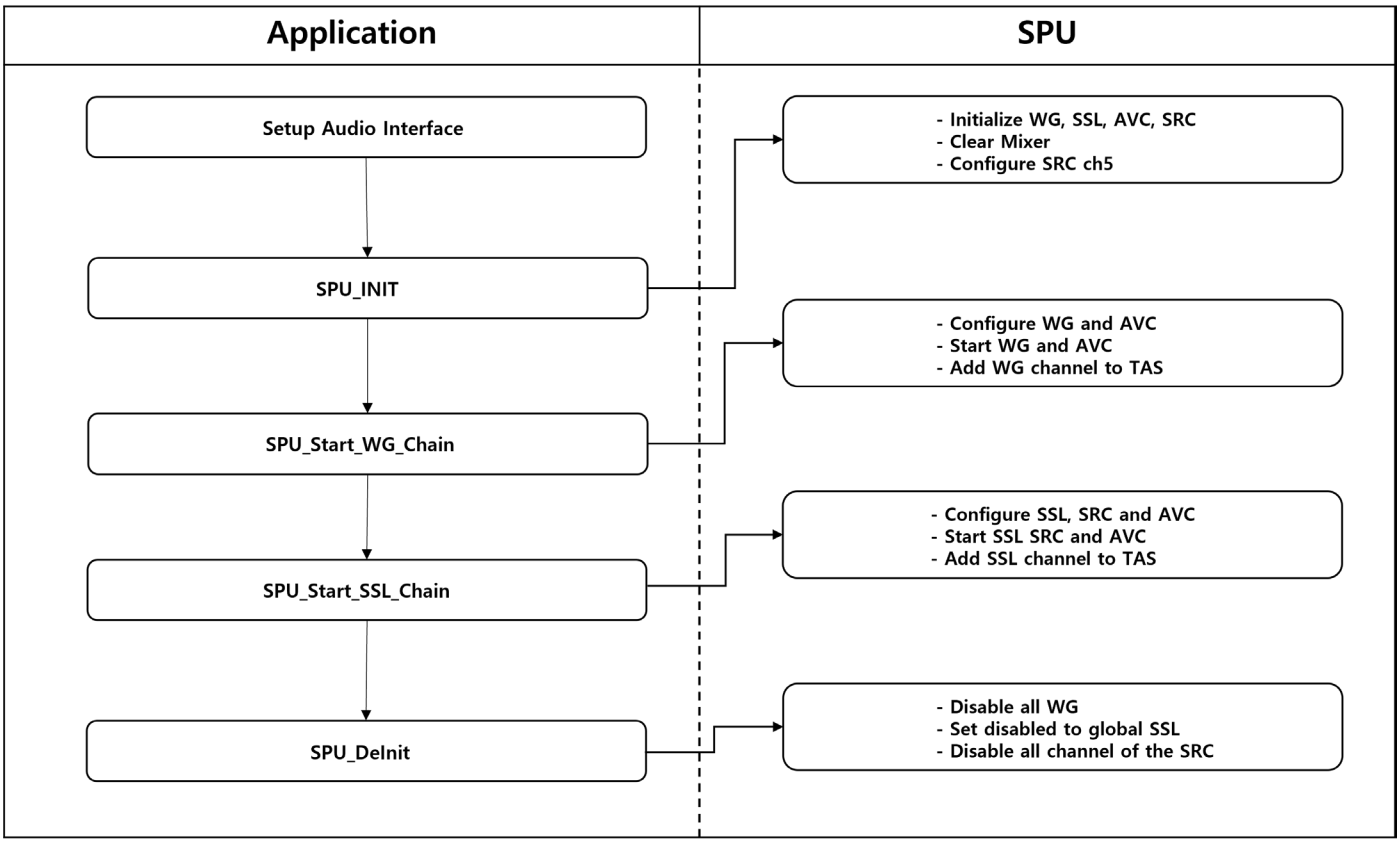

Figure 7.1 Diagram for SPU Driver

8.2 Example: Full-Chain Test

```
static void SPU_Test_FullChain
(
    void
)
{
    set_audio_interface(); /* Set Audio Interface */

    SPU_Init((SRCConfig_t*)&sSRCConigTable); /* Initialize SPU */

    /* WG0 ~ WG4 Configure and Start */
    SPU_Start_WG_Chain(SPU_WG_0, (WGConfig_t*)&sWGConigTable, (AVCConfig_t*)&sAVCConigTable);
    SPU_Start_WG_Chain(SPU_WG_1, (WGConfig_t*)&sWGConigTable, (AVCConfig_t*)&sAVCConigTable);
    SPU_Start_WG_Chain(SPU_WG_2, (WGConfig_t*)&sWGConigTable, (AVCConfig_t*)&sAVCConigTable);
    SPU_Start_WG_Chain(SPU_WG_3, (WGConfig_t*)&sWGConigTable, (AVCConfig_t*)&sAVCConigTable);
    SPU_Start_WG_Chain(SPU_WG_4, (WGConfig_t*)&sWGConigTable, (AVCConfig_t*)&sAVCConigTable);

    /* SSL0 ~ SSL4 Configure and Start */
    SPU_Start_SSL_Chain(SPU_SSL_0,
                        (SSLConfig_t*)&SSLConigTable,
                        (SRCConfig_t*)&sSRCConigTable,
                        (AVCConfig_t*)sAVCConigTable);
    SPU_Start_SSL_Chain(SPU_SSL_1,
                        (SSLConfig_t*)&SSLConigTable,
                        (SRCConfig_t*)&sSRCConigTable,
                        (AVCConfig_t*)sAVCConigTable);
    SPU_Start_SSL_Chain(SPU_SSL_2,
                        (SSLConfig_t*)&SSLConigTable,
                        (SRCConfig_t*)&sSRCConigTable,
                        (AVCConfig_t*)sAVCConigTable);
    SPU_Start_SSL_Chain(SPU_SSL_3,
                        (SSLConfig_t*)&SSLConigTable,
                        (SRCConfig_t*)&sSRCConigTable,
                        (AVCConfig_t*)sAVCConigTable);
    SPU_Start_SSL_Chain(SPU_SSL_4,
                        (SSLConfig_t*)&SSLConigTable,
                        (SRCConfig_t*)&sSRCConigTable,
                        (AVCConfig_t*)sAVCConigTable);

    /* Stop SSL Chains */
    (void) SAL_TaskSleep(5000UL);
    SPU_Stop_SSL_Chain(SPU_SSL_0, SSL_FORCE_STOP);
    SPU_Stop_SSL_Chain(SPU_SSL_1, SSL_FORCE_STOP);
    SPU_Stop_SSL_Chain(SPU_SSL_2, SSL_FORCE_STOP);
    SPU_Stop_SSL_Chain(SPU_SSL_3, SSL_FORCE_STOP);
    SPU_Stop_SSL_Chain(SPU_SSL_4, SSL_FORCE_STOP);

    /* Stop WG Chains */
    (void) SAL_TaskSleep(5000UL);
    SPU_Stop_WG_Chain(SPU_WG_0);
    SPU_Stop_WG_Chain(SPU_WG_1);
    SPU_Stop_WG_Chain(SPU_WG_2);
    SPU_Stop_WG_Chain(SPU_WG_3);
    SPU_Stop_WG_Chain(SPU_WG_4);

    SPU_DeInit();
}
```

9 REFERENCES

[1] TCC70xx Full Specification

[2] Contact Telechips for more details: sales@telechips.com

10 REVISION HISTORY

Rev. 0.10: 2022-09-21

- Preliminary version

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.
No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.
This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.
Telechips, Inc. cannot ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental, or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:
"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications (via internet, intranets and/or other networks), other content distribution systems (pay-audio or audio-on-demand applications and the like) or on physical media (compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use other firmware of mp3:
"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit http://mp3licensing.com".

For customers who use Digital Wave DRA solution:
"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:
"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:
"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:
"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:
"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."